

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO LAB 04

Đề tài: Mã hóa dữ liệu từ Client trước khi lưu xuống CSDL

Môn học: Bảo mật cơ sở dữ liệu

Sinh viên thực hiện:

Lưu Đức Toàn 23120095

Nguyễn Xuân Duy 23120121

Đào Nguyễn Nguyên 23120306

Giáo viên hướng dẫn:

Võ Hoàng Phúc Khang

Ngày 30 tháng 5 năm 2026

Mục lục

I	Giới thiệu	3
1	Bảng phân công việc	3
2	Tổng quan bài lab	3
II	Nội dung thực hành	4
1	Phương pháp xây dựng Stored Procedure	4
1.1	Quy trình xây dựng Stored Procedure thêm mới dữ liệu	4
1.2	Quy trình xây dựng Stored Procedure truy vấn dữ liệu	4
1.3	Đánh giá phương pháp	5
2	Cơ chế bảo mật	5
2.1	Cơ chế băm mật khẩu (Password Hashing)	5
2.2	Cơ chế tái tạo khóa bất định (Deterministic RSA 2048 Key Generation)	6
3	Màn hình quản lý đăng nhập	7
3.1	Cơ chế xác thực	7
3.2	Cơ chế tái tạo khóa và giải mã Lương	7
4	Màn hình quản lý nhân viên	7
4.1	Thêm mới nhân viên và khởi tạo khóa RSA	7
5	Màn hình quản lý nhân viên	8
5.1	Các Stored Procedure xử lý	8
5.2	Quy trình xử lý nghiệp vụ: Thêm, Sửa, Xóa	8
	Quy trình Thêm nhân viên:	8
	Quy trình Sửa nhân viên:	9
	Quy trình Xóa nhân viên:	9
6	Màn hình quản lý sinh viên và nhập điểm thi	9
6.1	Chức năng Thêm sinh viên (Bảo mật mật khẩu)	9
6.2	Chức năng nhập bảng điểm (Bảo mật điểm thi)	10
7	Theo dõi và Đánh giá bảo mật bằng SQL Server Profiler	10
7.1	Hướng dẫn cấu hình SQL Server Profiler	10
7.2	Kết quả theo dõi qua SQL Server Profiler	11
7.3	Nhận xét	11

Tài liệu tham khảo

13

I Giới thiệu

1 Bảng phân công việc

STT	MSSV	Phân công	Hoàn thành
1	23120095	d 2 ý đầu, e	100%
2	23120121	a, b	100%
3	23120306	d 3 ý sau	100%

2 Tổng quan bài lab

Trong bối cảnh an toàn thông tin ngày càng trở nên quan trọng, việc bảo vệ dữ liệu nhạy cảm trong các hệ thống cơ sở dữ liệu là một yêu cầu thiết yếu. Bài lab này tập trung vào việc tìm hiểu và ứng dụng các kỹ thuật mã hóa dữ liệu sử dụng thuật toán mã hóa công khai, tiêu biểu là thuật toán RSA (Rivest–Shamir–Adleman).

Mục tiêu chính của bài thực hành chính là nắm được quy trình triển khai mã hóa trong môi trường cơ sở dữ liệu, từ việc tạo và quản lý khóa, đến việc áp dụng mã hóa dữ liệu và xây dựng các cơ chế truy vấn phù hợp. Cụ thể, bài lab bao gồm các nội dung chính như:

- Tạo cặp khóa công khai và khóa bí mật phục vụ cho việc mã hóa và giải mã dữ liệu.
- Xây dựng bảng dữ liệu chứa các thông tin cần được bảo vệ và thực hiện mã hóa dữ liệu trước khi lưu trữ.
- Áp dụng thuật toán RSA để đảm bảo dữ liệu được mã hóa an toàn, chỉ có thể giải mã bởi các đối tượng được cấp quyền.
- Xây dựng stored procedure để truy vấn và xử lý dữ liệu đã mã hóa, đảm bảo tính bảo mật nhưng vẫn đáp ứng được yêu cầu khai thác thông tin.

Thông qua bài lab, không chỉ hiểu rõ hơn về nguyên lý hoạt động của mã hóa công khai mà còn biết cách ứng dụng vào thực tế, đặc biệt trong việc bảo vệ dữ liệu trong hệ quản trị cơ sở dữ liệu. Đây là nền tảng quan trọng để phát triển các hệ thống thông tin an toàn và đáng tin cậy trong môi trường thực tế.

II Nội dung thực hành

1 Phương pháp xây dựng Stored Procedure

Trong bài lab này, hai Stored Procedure được xây dựng nhằm phục vụ cho việc thêm mới và truy vấn dữ liệu nhân viên trong môi trường có áp dụng các cơ chế bảo mật dữ liệu. Việc mã hóa và băm dữ liệu được thực hiện phía client trước khi gửi lên hệ quản trị cơ sở dữ liệu nhằm giảm thiểu rủi ro lộ lọt thông tin nhạy cảm.

1.1 Quy trình xây dựng Stored Procedure thêm mới dữ liệu

Đối với chức năng thêm mới nhân viên, hệ thống thực hiện các bước chính như sau:

- **Tiếp nhận dữ liệu đầu vào:** Người dùng cung cấp các thông tin bao gồm mã nhân viên, họ tên, email, tên đăng nhập, mật khẩu và lương.
- **Băm mật khẩu phía client:** Trước khi gửi lên cơ sở dữ liệu, mật khẩu được băm bằng thuật toán SHA1. Kết quả băm được truyền vào tham số MK của Stored Procedure.
- **Tạo cặp khóa RSA phía client:** Ứng dụng phía client sinh cặp khóa RSA 2048 bit gồm khóa công khai (Public Key) và khóa bí mật (Private Key). Khóa công khai được gửi lên hệ thống thông qua tham số PUB.
- **Mã hóa dữ liệu lương:** Giá trị lương được mã hóa bằng thuật toán RSA 2048 sử dụng khóa công khai được tạo ở phía client. Dữ liệu sau mã hóa được truyền vào tham số LUONG.
- **Lưu trữ dữ liệu:** Stored Procedure thực hiện lưu trực tiếp các dữ liệu đã được xử lý bảo mật vào bảng NHANVIEN, bao gồm mật khẩu đã băm, lương đã mã hóa và khóa công khai tương ứng với nhân viên.

1.2 Quy trình xây dựng Stored Procedure truy vấn dữ liệu

Đối với chức năng truy vấn thông tin nhân viên, hệ thống thực hiện theo các bước sau:

- **Tiếp nhận thông tin truy vấn:** Người dùng cung cấp tên đăng nhập và mật khẩu thông qua các tham số của Stored Procedure.

- **Truy xuất dữ liệu:** Hệ thống tìm kiếm thông tin nhân viên dựa trên tên đăng nhập được cung cấp.
- **Lấy dữ liệu lương đã mã hóa:** Giá trị lương được đọc trực tiếp từ cơ sở dữ liệu ở dạng dữ liệu mã hóa RSA, không thực hiện giải mã trên SQL Server.
- **Trả kết quả:** Stored Procedure trả về các thông tin gồm mã nhân viên, họ tên, email và giá trị lương ở dạng mã hóa. Việc giải mã (nếu cần) sẽ được thực hiện phía client bằng khóa bí mật tương ứng.

1.3 Đánh giá phương pháp

Phương pháp triển khai mang lại một số lợi ích về bảo mật như sau:

- Mật khẩu không được lưu dưới dạng văn bản thuần (Plain Text) mà được băm bằng thuật toán SHA1 trước khi lưu trữ.
- Dữ liệu lương được mã hóa bằng RSA 2048 trước khi gửi lên cơ sở dữ liệu, giúp bảo vệ thông tin nhạy cảm ngay cả khi dữ liệu bị truy cập trái phép.
- SQL Server không lưu trữ khóa bí mật, do đó không thể tự giải mã dữ liệu lương nếu không có khóa tương ứng từ phía client.
- Việc lưu trữ khóa công khai trong thuộc tính PUBKEY giúp liên kết dữ liệu mã hóa với nhân viên tương ứng và hỗ trợ việc quản lý khóa trong hệ thống.

2 Cơ chế bảo mật

Điểm khác biệt cốt lõi của bài thực hành này so với kiến trúc Lab 03 cũ là sự dịch chuyển toàn bộ tác vụ mật mã từ Cơ sở dữ liệu (Database-side) lên Tầng ứng dụng (Client-side). Cơ sở dữ liệu giờ đây bị "mù" (Blind) hoàn toàn trước dữ liệu nhạy cảm và chỉ đóng vai trò là một kho lưu trữ các khối ciphertext.

2.1 Cơ chế băm mật khẩu (Password Hashing)

Mật khẩu của người dùng được xử lý băm một chiều ngay tại tầng ứng dụng Python thông qua hàm `hash_password_sha1()` sử dụng thuật toán thô SHA1 từ thư viện `pycryptodome`:

- Chuỗi mật khẩu rõ (Plaintext) đi qua bộ băm sinh ra một chuỗi ký tự định dạng Hex viết hoa dài 40 ký tự độc nhất thông qua phương thức `hexdigest().upper()`.
- Kết quả chuỗi Hex băm này sau đó được chuyển đổi về dạng nhị phân (`bytearray`) để truyền trực tiếp xuống lưu trữ trong cột `MATKHAU` kiểu `VARBINARY(MAX)` của cơ sở dữ liệu. Cơ chế này đảm bảo mật khẩu rõ không bao giờ xuất hiện trên đường truyền mạng hay dưới dạng plaintext trong Database.

2.2 Cơ chế tái tạo khóa tất định (Deterministic RSA 2048 Key Generation)

Để loại bỏ rủi ro khi lưu trữ private key, bài thực hành Lab 04 triển khai giải pháp **Sinh khóa tất định (Deterministic Key Generation)** tại tầng ứng dụng Python. Cơ chế này cho phép khôi phục chính xác cùng một cặp khóa RSA 2048-bit trên bộ nhớ RAM mỗi khi giảng viên nhập đúng mật khẩu đăng nhập, dựa trên quy trình tối ưu sau:

- **Khởi tạo khóa môi (Seed Generation):** Ứng dụng thu thập Mật khẩu rõ (`password`) và Mã nhân viên (`manv`) từ màn hình đăng nhập. Để triệt tiêu lỗi hỏng đụng độ chuỗi nối (Concatenation Collision), hệ thống áp dụng ký tự phân cách `'|'` để tạo ra chuỗi nguyên liệu duy nhất dạng `f"{password}|{manv}"`, sau đó băm qua hàm `SHA256` thành khối dữ liệu nhị phân `seed` cố định 32 bytes (256 bits).
- **Cấu hình bộ sinh số tất định (DRBG):** Để cung cấp hàm cấp phát byte liên tục (`randfunc`) cho lệnh sinh khóa `RSA.generate()`, ứng dụng cấu hình thuật toán `AES-256` ở chế độ bộ đếm (Counter Mode - `AES.MODE_CTR`) với khóa mật mã chính là khối `seed` vừa tạo. Hàm ngẫu nhiên tùy biến `deterministic_randfunc(n)` sẽ liên tục mã hóa các khối byte rỗng `b'\x00'` để tạo ra một dòng bit giả ngẫu nhiên có tính tất định, trùng khớp hoàn toàn qua mọi lần thực thi.
- **Đóng gói và xuất định dạng khóa (Key Exporting):** Cặp khóa sau khi tính toán được đóng gói theo chuẩn `PEM (Privacy-Enhanced Mail)`. Khóa công khai (Public Key `PEM`) được giải mã sang dạng văn bản thô (`.decode('utf-8')`) để lưu trữ tại cột `PUBKEY` kiểu `VARCHAR(MAX)` dưới Database phục vụ cho việc mã hóa điểm. Ngược lại, Khóa bí mật (Private Key `PEM`) chỉ tồn tại tạm thời trong RAM ở dạng biến cục bộ và sẽ giải phóng ngay sau khi kết thúc tiến trình giải mã.

3 Màn hình quản lý đăng nhập

Màn hình đăng nhập được thiết kế với giao diện trực quan nhằm tiếp nhận Tên đăng nhập và Mật khẩu của nhân viên.

3.1 Cơ chế xác thực

Khi nhân viên thực hiện đăng nhập, mật khẩu nhập sẽ được băm bằng thuật toán **SHA1** thông qua hàm `hash_password_sha1(matkhau)`. Ứng dụng gọi thủ tục `SP_SEL_PUBLIC_ENCRYPT_NHANVIEN` và truyền tham số Tên đăng nhập cùng Mật khẩu đã băm (`MATKHAU_HASH`).

3.2 Cơ chế tái tạo khóa và giải mã Lương

Nếu xác thực thành công, CSDL trả về thông tin nhân viên, bao gồm mức Lương đã được mã hóa bằng RSA. Quá trình xử lý tiếp theo diễn ra hoàn toàn tại Client:

1. **Tạo Private Key:** Ứng dụng sử dụng thuật toán tạo khóa RSA (2048-bit) có tính tất định (Deterministic RSA Key Generation) dựa trên Mật khẩu (dưới dạng Plaintext mà người dùng vừa nhập) và Mã nhân viên.
2. **Giải mã Lương:** Sử dụng Private Key vừa sinh, giải mã dữ liệu Lương từ dạng Ciphertext sang Plaintext.
3. **Hiển thị & Xóa khóa:** Mức lương được hiển thị trên giao diện, đồng thời Private Key không bao giờ được lưu xuống đĩa cứng, đảm bảo an toàn tuyệt đối ngay cả khi máy tính bị xâm nhập sau này.

4 Màn hình quản lý nhân viên

Màn hình quản lý nhân viên (chỉ dành cho Admin) cho phép Thêm, Sửa, Xóa thông tin các nhân viên khác trong hệ thống.

4.1 Thêm mới nhân viên và khởi tạo khóa RSA

Quá trình thêm nhân viên yêu cầu quản trị viên nhập đầy đủ thông tin, đặc biệt là Lương cơ bản và Mật khẩu khởi tạo. Ứng dụng thực hiện quy trình sau:

- **Sinh cặp khóa RSA:** Dựa trên Mật khẩu khởi tạo và Mã nhân viên mới, một cặp khóa (Public Key và Private Key) được sinh ra.
- **Mã hóa Lương cơ bản:** Mức lương nhập vào được mã hóa ngay lập tức bằng Public Key.
- **Băm mật khẩu:** Mật khẩu khởi tạo được băm bằng thuật toán SHA1.
- **Lưu trữ dữ liệu:** Ứng dụng truyền các tham số bao gồm: Mã nhân viên, Họ tên, Email, Tên đăng nhập, **Mật khẩu (đã băm)**, **Lương (đã mã hóa)**, và **Public Key** xuống thủ tục SP_INS_PUBLIC_ENCRYPT_NHANVIEN. Khóa công khai (Public Key) được lưu vào CSDL để phục vụ mã hóa sau này, trong khi Private Key bị hủy bỏ lập tức.

5 Màn hình quản lý nhân viên

Màn hình quản lý nhân viên (chỉ dành cho Admin) cho phép thao tác quản lý danh sách nhân viên trong hệ thống.

5.1 Các Stored Procedure xử lý

Hệ thống sử dụng các Stored Procedure sau để tương tác với cơ sở dữ liệu ở chức năng quản lý nhân viên:

- SP_SEL_NHANVIEN: Truy xuất danh sách toàn bộ nhân viên trong hệ thống để hiển thị (không bao gồm Lương hay Mật khẩu chưa mã hóa).
- SP_INS_PUBLIC_ENCRYPT_NHANVIEN: Thêm mới một nhân viên. Yêu cầu quyền Admin (ISADMIN = 1).
- SP_UPD_NHANVIEN: Cập nhật thông tin nhân viên (Họ tên, Email, Tên đăng nhập, Lương đã mã hóa). Yêu cầu quyền Admin.
- SP_DEL_NHANVIEN: Xóa nhân viên khỏi hệ thống (Admin không được tự xóa chính mình). Yêu cầu quyền Admin.

5.2 Quy trình xử lý nghiệp vụ: Thêm, Sửa, Xóa

Quy trình Thêm nhân viên: Quá trình thêm yêu cầu quản trị viên nhập thông tin nhân viên, đặc biệt là Lương cơ và Mật khẩu. Ứng dụng thực hiện:

- **Sinh cặp khóa RSA:** Dựa trên Mật khẩu khởi tạo (mặc định lấy Mã nhân viên) và Mã nhân viên mới, một cặp khóa được sinh ra.
- **Mã hóa Lương cơ bản:** Mức lương nhập vào được mã hóa tại Client bằng Public Key.
- **Băm mật khẩu:** Mật khẩu khởi tạo được băm bằng thuật toán SHA1.
- **Lưu trữ dữ liệu:** Gọi thủ tục SP_INS_PUBLIC_ENCRYPT_NHANVIEN để truyền tham số xuống CSDL (Mật khẩu đã băm, Lương đã mã hóa, Public Key). Private Key lập tức bị hủy bỏ.

Quy trình Sửa nhân viên: Admin chọn nhân viên cần cập nhật và nhập dữ liệu mới:

- Nếu có cập nhật mức lương, ứng dụng truy xuất **Public Key** của nhân viên đó để thực hiện mã hóa lại mức lương bằng RSA tại Client.
- Gọi thủ tục SP_UPD_NHANVIEN để cập nhật các thông tin mới vào CSDL.

Quy trình Xóa nhân viên: Admin chọn nhân viên cần xóa và nhấn xác nhận:

- Ứng dụng gọi thủ tục SP_DEL_NHANVIEN truyền vào Mã nhân viên cần xóa và Mã Admin.
- Hệ thống tự động xác thực quyền và xóa nhân viên tương ứng khỏi CSDL.

6 Màn hình quản lý sinh viên và nhập điểm thi

Màn hình này cho phép nhân viên thao tác trên dữ liệu sinh viên của những lớp mà mình quản lý, đáp ứng yêu cầu phân quyền mức dữ liệu (Row-level Security).

6.1 Chức năng Thêm sinh viên (Bảo mật mật khẩu)

Chức năng "Thêm sinh viên" được tái cấu trúc theo mô hình mã hóa tại Client, biến Database thành một kho lưu trữ thô và bảo vệ dữ liệu tuyệt đối trên đường truyền mạng:

- **Xử lý băm tại Client (Băm mật khẩu):** Khi giảng viên thực hiện thêm một sinh viên mới, chuỗi mật khẩu rõ (Plaintext) nhập từ giao diện sẽ ngay lập tức được ứng dụng Python gọi hàm `hash_password_sha1()` để thực hiện băm một chiều mật khẩu bằng thuật toán SHA1.

- **Ảnh xạ kiểu dữ liệu nhị phân an toàn:** Kết quả từ bộ băm là một chuỗi ký tự dạng mã Hex. Để truyền xuống các cột dữ liệu, ứng dụng chuyển đổi kết quả băm về dạng mảng nhị phân (bytearray). Khối bytearray này được gán làm tham số @MATKHAU_HASH và đẩy trực tiếp xuống thủ tục SP_INS_SINHVIEN để ghi vào cột MATKHAU (VARBINARY(MAX)).

6.2 Chức năng nhập bảng điểm (Bảo mật điểm thi)

Khi nhân viên chọn một sinh viên và mở chức năng "Quản lý Điểm", ứng dụng hiển thị một giao diện nhập điểm thi cho từng học phần:

- **Mã hóa điểm thi:** Trước khi gửi lệnh lưu điểm, ứng dụng truy xuất **Public Key của chính nhân viên đang đăng nhập** (đã được lấy về và lưu trong Session từ quá trình đăng nhập).
- Mức Điểm thi (ví dụ: 8.5) được chuyển sang dạng chuỗi (String) và mã hóa bằng thuật toán RSA sử dụng Public Key này.
- Dữ liệu sau khi mã hóa (DIEMTHI_ENCRYPT ở dạng mảng byte) được truyền cho thủ tục SP_INS_BANGDIEM kèm theo Mã SV và Mã HP.
- Cơ chế này đảm bảo rằng Điểm thi được an toàn trên đường truyền và chỉ có nhân viên quản lý (sở hữu Private Key tương ứng với Public Key đã dùng) mới có khả năng cung cấp Mật khẩu cá nhân để giải mã và xem lại mức điểm này trong tương lai.

7 Theo dõi và Đánh giá bảo mật bằng SQL Server Profiler

Trong phần này, công cụ SQL Server Profiler được sử dụng để theo dõi các câu lệnh truyền từ ứng dụng Python xuống cơ sở dữ liệu khi thực hiện chức năng thêm bảng điểm, nhằm kiểm chứng tính an toàn của luồng thực thi và cơ chế mã hóa.

7.1 Hướng dẫn cấu hình SQL Server Profiler

- **Khởi tạo Trace:** Từ SQL Server Management Studio (SSMS), mở Tools → SQL Server Profiler và tạo một Trace mới.
- **Thiết lập Event Selection:** Để tránh nhiễu thông tin, tiến hành lọc các sự kiện không cần thiết. Chỉ chọn các sự kiện quan trọng thuộc nhóm *Stored Procedures* và *TSQL* bao gồm:

RPC:Completed (bắt các Remote Procedure Call từ pyodbc) và **SP:Completed** (bắt trạng thái hoàn thành của Stored Procedure).

- **Thực thi theo dõi:** Nhấn Run để Profiler bắt đầu lắng nghe. Trở lại giao diện ứng dụng Python, tiến hành đăng nhập với tài khoản Giảng viên, chọn một sinh viên và thực hiện chức năng nhập điểm. Sau khi nhấn "Lưu Điểm", tạm dừng (Pause) Profiler để phân tích dữ liệu.

7.2 Kết quả theo dõi qua SQL Server Profiler

Kết quả Trace ghi nhận được dòng sự kiện **RPC:Completed** khi ứng dụng gọi các Stored Procedure thực hiện thêm nhân viên hoặc lưu bảng điểm:

EventClass	TextData	ApplicationName	NTUserName	LoginName	CPU	Reads	Writes	Duration	ClientProcessID	SPID	StartTime	EndTime
RPC:Completed	declare @p1 int set @p1=1 exec sp_...	Python	sa	sa	15	658	1	23	14196	98	2026-05-30 11:11:56...	2026-05-30 11:11
RPC:Completed	exec sp_unprepare 1	Python	sa	sa	0	0	0	0	14196	98	2026-05-30 11:11:56...	2026-05-30 11:11


```

declare @p1 int
set @p1=1
exec sp_preexec @p1 output, N'@P1 nvarchar(8),@P2 nvarchar(8),@P3 varbinary(256),@P4 nvarchar(8)', N'EXEC SP_INS_BANGDIEM
@P1,@P2,@P3,@P4', N'SV01', N'HP02', 0x3AFC6E2147A48D7727478A5A7096B0927B1B1054F5133E38CC37031C6EC75049B60D093C67884F8D83E5F229078F5442500C570E2D07229FE0CB81434962390904EE90C2EC62142BF17889AD8A837470506C68C12DB04
select @p1
go
exec sp_unprepare 1
go

```

- **Sự kiện RPC:Completed:** Hiện thị câu lệnh `declare @p1 int set @p1=1 exec sp_preexec...`. Điều này chứng tỏ ứng dụng Python sử dụng cơ chế prepare statement để chuẩn bị câu lệnh và truyền tham số một cách an toàn.
- **Sự kiện SP:Completed:** Ghi nhận lệnh gọi `EXEC SP_INS_BANGDIEM @P1, @P2, @P3, @P4`. Điểm khác biệt là tham số Điểm thi (@P3) không còn là plaintext mà là một chuỗi hex chứng minh dữ liệu đã được mã hóa bằng Public Key ngay tại Client trước khi truyền đi.

7.3 Nhận xét

Dựa trên kết quả trace từ Profiler, ta có thể rút ra các nhận xét sau về khía cạnh bảo mật:

- **Phòng chống SQL Injection:** Việc ứng dụng (qua thư viện pyodbc) không gửi các lệnh INSERT/UPDATE thô mà sử dụng Remote Procedure Call với tham số hóa (Parameterized queries) đã giúp hệ thống ngăn chặn hoàn toàn các cuộc tấn công SQL Injection từ phía Client.
- **Mã hóa điểm thi (Data at Rest):** Dữ liệu lưu vào CSDL là các chuỗi byte nhị phân mã hóa (Ciphertext). Ngay cả Quản trị viên CSDL (DBA) cũng không thể đọc được nội dung thật vì Private Key chỉ được sinh ra và sử dụng tạm thời tại Client.

- **Bảo mật trên đường truyền (Data in Transit):** Khác biệt cốt lõi so với Lab 03 là dữ liệu đã được mã hóa bằng Public Key RSA **trước khi** gửi đi. Việc điểm số truyền qua mạng dưới dạng Ciphertext giúp khắc phục triệt để rủi ro tấn công Man-in-the-Middle (MITM) mà không bắt buộc cấu hình thêm SSL/TLS ở tầng mạng.

Tài liệu tham khảo